COMP3310/ENGN3539/etc Assignment 3 – Testing MQTT

Context:

- This assignment is worth 15% of the final mark
- It is due by **Friday 29 May 2026 23.59 AEST**
- Late submissions will not be accepted, except in special circumstances. *Any extensions should be requested well before the due date, with appropriate evidence. Please use the extension-request link on Canvas and not direct emails.*
- *This is another evolving experiment for us, any issues/corrections will be announced via canvas.*

Assignment 3

MQTT is the most common open IoT protocol being deployed today. It uses a publisher/subscriber model, allowing for an almost-unbounded number of sources to publish information, each at their own rate, and subscribers to receive information as desired. As such, it is designed to provide high-performance communication mechanisms, with minimal delays and excellent scaling performance. We'll use it to monitor the performance of some imaginary system: say counting the total kilograms of minerals rushing by on conveyor belts, that you can control. This assignment will look at the functionality and performance of the publishers, brokers, the network and subscribers.

This is a coding, analysis and writing assignment. You may code in C/Java/Python or any programming language that a tutor can assess (hope that's enough for everyone), and yes, you may use MQTT and other common helper libraries. The assessment will not rely solely on running on your code, but more on the data gathering and your analysis. You need to note in your report/code any libraries you are using.

Submitting

You will be submitting two things: your code and your analysis report. Note that there will be two submission links on the Canvas course-site:

1. Your code must be submitted to Canvas as a zip file, with instructions in your report on how to compile/run the components as appropriate.
2. Your analysis report (pdf) must be submitted via TurnItIn on the canvas site, so ensure you quote and cite sources properly.

Outcomes

We're assessing your understanding of MQTT, as well as your code's functionality in subscribing and publishing to a broker, dealing with high message rates, measuring message performance and statistics of a networked application, and your insight to interpret what you're seeing. You will be exploring MQTT functionality, the quality-of-service (QoS) levels, describing how they work (or not), why you would use different levels, and how they perform in real-world deployments.

Resources

You will need to set up your own MQTT server/broker for you to connect to as per the specifications below, on a remote computer[1] and the further that is away from Canberra, the better. It needs to be able to handle a significant volume of traffic/transactions, and for you to run a publisher alongside the broker. Amazon Web Services (AWS) is recommended[2], with an instance outside of Australia, but there are plenty

[1] Hosted mqtt brokers are going to be highly throttled, or may kick you off for excessive traffic, and they won't let you run code alongside them.

[2] We'll post a quick AWS guide to Canvas, though most tutors can demonstrate. It will just need a basic (free) hosted Linux image, natively running your MQTT broker of choice, and your publisher.

of other cloud providers that offer free VMs. There are several free brokers to choose from, https://mqtt.org/software/ has a good list. FlashMQ appears to be very fast, and Mosquitto (clients) should be familiar from the tutorial. Some don't offer all QoS levels, or share a lot of $SYS information, or have other limitations, so read ahead before picking one.

You can test your broker works by subscribing to the $SYS/# topics, which describe the server, and it will help get you familiar with the information presented there - you will be using them for your analysis later.

Assignment programming:

After installing a broker and checking it, you need to write two programs.

- **A Publisher**: This will run on the same machine as the broker, to minimise the publishing performance uncertainty and hopefully achieve blazing performance.
    - A Publisher will first subscribe (listen) to a set of 'request' topics, namely **request/qos**, **request/delay,** and **request/messagesize**. You also want a **request/go** topic to receive a "start" value, to avoid the race condition when the request/# topics are updated. Based on these, it will then start publishing accordingly.
    - The Publisher will send a sequence of simple messages to the broker for 30 seconds. Each message will be of the form **counter:timestamp:xxx…xxx**, where:
        - **counter** is an incrementing value for each message (0, 1, 2, 3, …).
        - **timestamp** is the precise current local time (a number, millisecond level, or better).
        - **xxx…xxx** is a string of repeated characters ("x") of length <message-size>, which can have values of 1, 1000, and 4000.
    - The Publisher will publish those messages to the broker at a requested MQTT <QoS> level (0, 1 or 2), with a requested <delay> between messages (either 0ms (no delay), or 100ms), and a message of the requested size.
    - The Publisher will publish to the topic **counter/<qos>/<delay>/<message-size>**, so e.g. **counter/0/100/4000** is the topic for messages coming from the Publisher at *qos=0*, *delay=100* and *message-size=4000*.
    - After it has finished its burst of messages, each Publisher should (a) tell the Analyser (see below) it has finished with a "done" message to **request/go**, and (b) go back to listening to the 'request' topics for the next round of instructions.
    - *At 0ms delay, qos=0 and messagesize=0 a publisher should be able to publish very quickly, likely many thousands of messages per second. At 100ms delay, it's just 10 messages per second, which should have no issues, and is useful to verify everything is working ok.*

- **An Analyser**: Who controls your Publisher? Your Analyser, running on your own computer locally.
    - Your Analyser will start by subscribing to the relevant **counter** topic(s) on the broker, then publishing to the **request/qos**, **request/delay,** and **request/messagesize,** and finally the **request/go** topics, asking for the Publisher to deliver accordingly.
    - It will then listen to the specified **counter** topic(s) on the broker and take measurements as below to report statistics on the performance of the publisher/broker/network/analyser combination.
    - The measurements should be taken across the various delay values (0,100ms), publisher-to-broker QoS (0,1,2), and message-size (1,1000,4000), so that you can compare them; things can get weird under load.
    - You will also need to run the full suite of tests with each of the three QoS values for the broker-to-analyser subscription, as things can also get weirder when the Publisher and Subscriber have very different QoS.
        - *You may need to disconnect and reconnect when changing the subscription QoS; this can be broker-specific behaviour.*

- Yes, that's 3*2*3*3=54 separate tests, each taking 30sec. Statistics for performance analysis requires a reasonable amount of data, and in a real-world test you should collect a lot more wit varying parameters, and over longer periods! Fortunately your code could do it all for you.

Data capture and Analysis

Once your code is working, you need to tackle the following:

Start the Publisher, then run your Analyser. Have the Analyser tell the broker what you want the Publisher(s) to send, and record data as below.

> *Tips: (i) only ever print to screen for debugging, not while actually publishing/measuring, otherwise it will slow your code down **a lot** and mess up your data. (ii) Use the counter values to tell you what messages are arriving, or are not arriving, to calculate the rates below, (iii) use the timestamps to calculate changes to the transmission delay, (iv) minimise any processing during the run, either on the publisher or the analyser – focus on collecting data (v) if you get very similar results across all combinations, check with wireshark what QoS is being used.*

Collect statistics, for each delay/QoS/message-size combination, to measure over the test period:

1. The total mean rate of messages you receive from the publisher across the period *[messages/second]*.

2. The percentage of
    a. Any message loss *(how many messages did you see, versus how many should you have seen)*
    b. Any out-of-order messages *(i.e. how often do you get a smaller number after a larger number)*
    c. Any duplicate messages *(i.e. how often do you see the same message published)*

3. The average inter-message-gap (timestamp difference) between consecutive messages and the standard-deviation over a run *[both in milliseconds]*. *Only measure for actually consecutive counter-value messages, ignore any values where you miss any messages in between.*

4. While measuring the above also subscribe to and record the $SYS/# measurements, and identify what, if anything, on the broker do any loss/misordered/duplicates correlate with. You can do this with a separate client or your own code. *(Look at measurements under e.g. 'load', 'heap', 'active clients', 'messages'; anything that seems relevant, and also check the broker configuration options and defaults. See e.g. https://mosquitto.org/man/mosquitto-8.html for ideas. Be aware of the timing and frequency of the $SYS measurements, to reflect when you actually put the broker under load. With some brokers you may be able to configure the frequency of $SYS reports.)*

Reporting

In your report: *[around 4-5 pages of text, plus any diagrams and charts]*

1. Subscribe to your broker to retrieve some $SYS/# value. Wireshark the handshake for one example of each of the differing QoS-levels (0,1,2), include screenshots in your report that show the wireshark capture of your subcription (filter for mqtt), and briefly explain how each QoS-level transfer works, and what it implies for message duplication and message order. Discuss briefly in your report in which circumstances would you choose each QoS level, on pub->broker and broker->sub. *[around 0.5 page of text]*

2. Summarise your measurements from above, in suitable table form, and with simple charts, to compare the impact of different message sizes, delays, and QoS combinations. Explain what you expected to see, especially in relation to the different QoS levels, and whether your expectations

were matched. Also describe what correlations of measured rates with $SYS topics you expected to see and why, and whether you do, or do not.

3. Consider the broader end-to-end (internet-wide) network environment, in a situation with millions of sensors publishing frequently to thousands of subscribers. Explain in your report *[around 1 page]*
    a. What performance challenges might be when using MQTT for extremely high volumes of messages, from the sources publishing their messages, all the way through the network and broker to your subscribing client application. If you lose messages, where might they be lost, and why? Think about links, routers, memory/buffers, cpus, long paths/high delays, layer 3/4/7, etc.
    b. How the different QoS levels may help, or not, in dealing with the challenges.
    c. Why 'retaining' messages would be a bad idea in this context.

Bonus questions:

Some extension questions to tackle, if you want and have time, for up to 3 bonus marks.

1. What results do you get with several (3 or many more) publishers with qos=0, delay=0ms. Keep in mind you can have multiple publishers listening and publishing in parallel. This will be subject to your host's performance, and you'll probably need to somehow make it clear to your analyser which message is coming from which publisher.
2. What happens if you run the same experiments with the publisher remote from the broker? It could be on another virtual machine instance in Amazon somewhere else in the world, or a(nother) computer back at your home with the Analyser.
3. What is the performance difference between your analyser subscribing to an explicit list of topics (counter/A/B/C) and subscribing to a top-level wildcard topic (counter/#). Why would/could it make a difference?

Assessment

Leave your broker running after submission, so we can do a quick test to check it for marking.

We'll be marking out of 15 marks:

- Your code clarity, and code-documentation/comments (3.5 marks)
    - *Could somebody else pick this up, debug it or add features? Do you explain your approach and choices you made? Can tutors run both programs easily?*
- Your code subscribing, properly publishing/listening to the broker, and collecting data (1.5 marks).
    - *Do your Analyser and Publisher work efficiently and as specified? Can your publishers publish messages at a high rate when the delay=0ms?*
- Your analysis report addressing the questions above (10 marks)
    - *Have you done the wireshark?*
    - *Did you neatly summarise the statistics you collected, to highlight and explain interesting results? Have you considered the whole workflow of data/messages from the publisher process to the analyser process, and all the various protocol overheads involved?*
    - *Be aware, all students will have different set-ups, and may see very different results.*